MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software and Automation Engineering

Tacu Loredana

FAF-242

REPORT

Laboratory Work No.1

The OS as a Resource Manager

Track A: the real OS, from the outside (Linux)

Checked by:

Reițman Patricia

Chisinau – 2026

1 Purpose of the Laboratory Work

The goal of this laboratory work is to observe the operating system, from the outside, acting as a resource manager on a provided Linux virtual machine. Specifically, we must watch the four core jobs of any OS – managing files, processes, memory, and devices – happen live in the terminal, and record what is observed rather than only which commands were typed.

2 Problem Conditions

Requirements and Conditions:

(1) Log in to the provided Linux VM, create a working folder (~/os-lab1), and record observations as commands are run.

(2) **Part 1 – Files and directories:** navigate the filesystem tree, create/copy/rename/remove a file, and inspect ownership and permissions with chmod.

(3) **Part 2 – Processes:** list running processes with ps, start and stop a background job, and inspect a live process through /proc.

(4) **Part 3 – Memory:** read total/used/free RAM and swap with free -h and /proc/meminfo, and check the resident memory of one process.

(5) **Part 4 – Devices and storage:** inspect mounted filesystems, block devices, and device files under /dev.

(6) For each part, answer the “Observe” questions and record one interesting command output.

3 Setup

```
root@ubuntu-4gb-hel1-1:~/os-lab1# mkdir -p ~/os-lab1 && cd ~/os-lab1
whoami
uname -a
uptime
root
Linux ubuntu-4gb-hel1-1 6.8.0-142-generic #142-Ubuntu SMP PREEMPT_DYNAMIC Wed
 Sep  2 14:24:27 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
 14:26:05 up 33 min,  2 users,  load average: 2.22, 2.13, 1.74
root@ubuntu-4gb-hel1-1:~/os-lab1#
```

Figure 1: whoami, uname -a, uptime on the provided VM.

4 Part 1 – Files and Directories

Navigated the tree with pwd, ls -la /, ls -la ~, and cd /etc && ls | head: 1

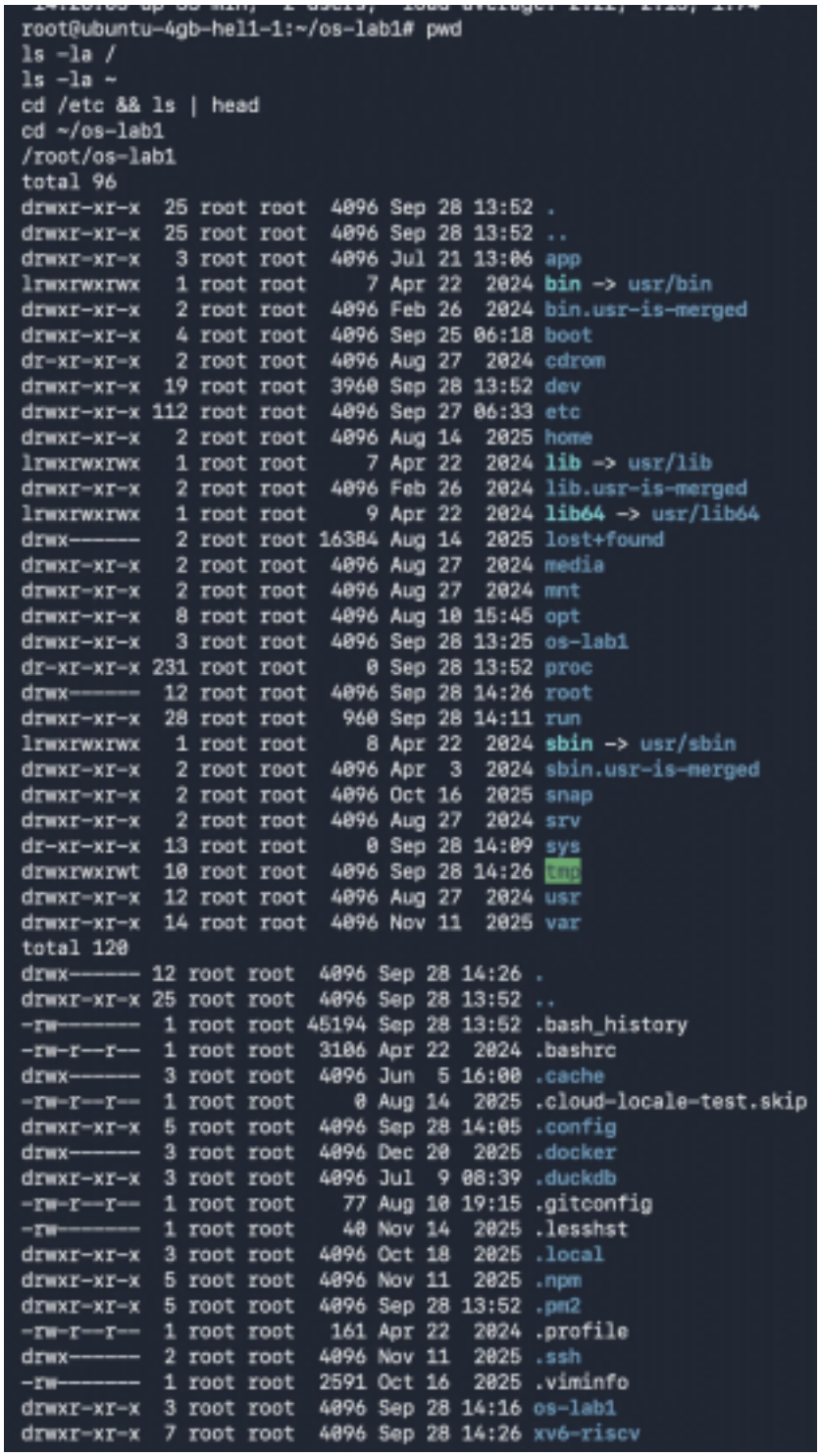


Figure 2: Root and home directory listings.

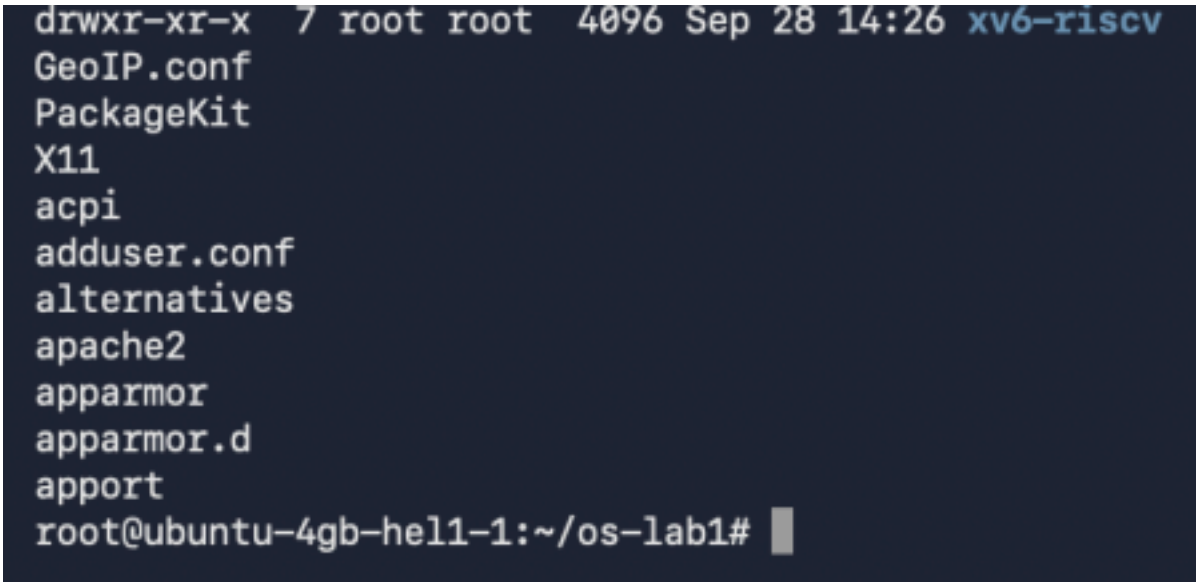


Figure 3: Tail of the /etc listing.

Created, copied, renamed, and removed a file:

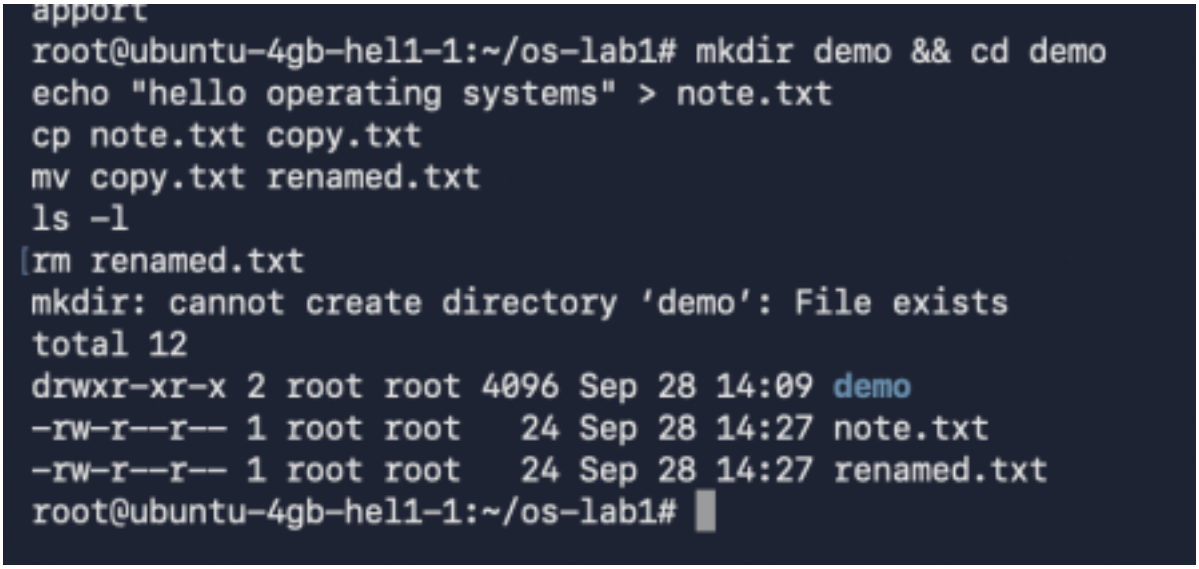


Figure 4: File operations in ~/os-lab1.

Note on this run: mkdir demo && cd demo had mkdir fail (demo already existed from an earlier attempt), so cd demo never ran because of &&. Every following command therefore executed directly in ~/os-lab1 instead of inside demo/, which is why note.txt and renamed.txt appear next to the demo directory above, not inside it.

Looked at ownership and permissions:

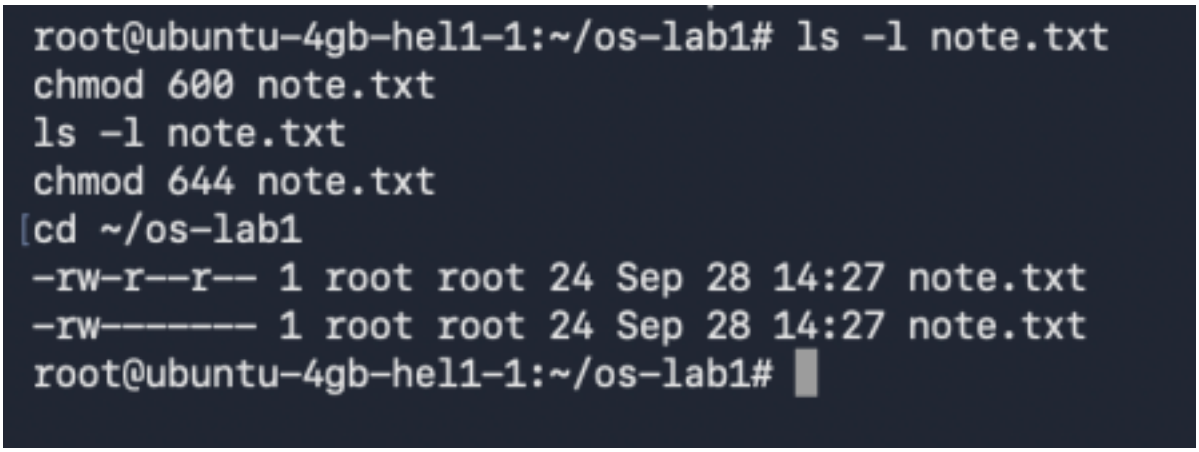


Figure 5: chmod 600 then chmod 644 on note.txt.

Observe

(1) **Owner/group of the created files:** root/root – files inherit the identity of the user (and primary group) that created them, here root since logged in as root on this VM.

(2) **The ten characters after chmod 600:** -rw-------. The leading - marks a regular file. The next three characters (rw-) are the owner's permissions: read and write, no execute. The remaining six characters are all -, meaning the group and everyone else have no access at all – only root (the owner) can read or write the file.

(3) **Two directories directly under /:**

- /etc – system-wide configuration files that programs read on startup.
- /var – variable data that changes while the system runs: logs, spool files, and in this case the Docker image/container layers under /var/lib/docker.

5 Part 2 – Processes

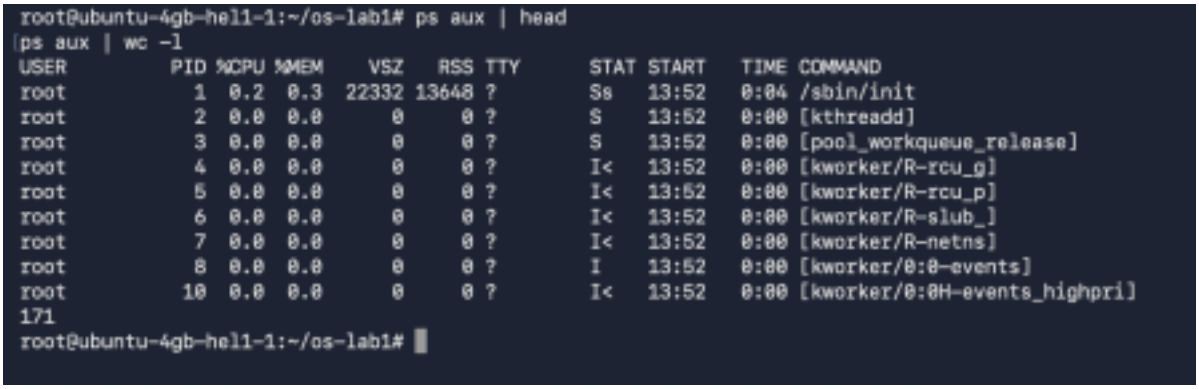


Figure 6: ps aux | head and ps aux | wc -l.

Started a background job, found it, and tried to stop it:

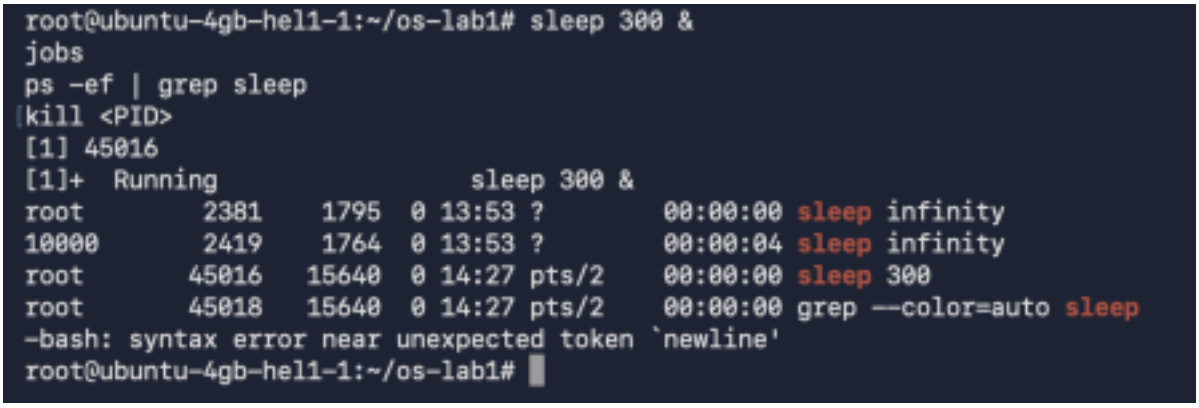


Figure 7: sleep 300 &, jobs, ps -ef | grep sleep.

Note on this run: the backgrounded sleep got PID 45016. Typing the kill command exactly as written in the handout, kill <PID>, instead of substituting the real number caused bash to read <PID> as an input redirection, printing a syntax error instead of killing anything. The correct command was kill 45016.

Peeked at /proc/<pid> for a fresh sleep 300:



Figure 8: /proc/$PID/ listing and status for the sleeping process.

Observe

(1) **PID of process 1:** 1, running /sbin/init (backed by systemd, confirmed via ps -p 1 -o pid,comm,args → 1 systemd /sbin/init). It is the first process the kernel starts at boot, has no parent, and adopts any orphaned process, making it the ancestor of every other process on the system.

(2) **Roughly how many processes on the idle VM:** ps aux | wc -l reported 171 lines, i.e. about 170 processes once the header is discounted. The VM is not perfectly idle: it runs Docker with several overlay containers plus Apache and PM2-managed services, which explains a higher-than-minimal count.

(3) **State: line for the sleeping process:** S (sleeping) – exactly as expected. Once sleep calls into the kernel's nanosleep, it is taken off the run queue and marked as interruptibly sleeping until its timer expires or it receives a signal.

6 Part 3 – Memory

```
root@ubuntu-4gb-hel1-1:~/os-lab1# free -h
cat /proc/meminfo | head -6
sleep 300 &
PID=$!
grep VmRSS /proc/$PID/status
kill $PID
               total        used        free      shared  buff/cache   available
Mem:           3.7Gi       1.4Gi       449Mi       5.3Mi       2.2Gi       2.3Gi
Swap:             0B          0B          0B
[2]+  Terminated              sleep 300
MemTotal:        3911572 kB
MemFree:          460680 kB
MemAvailable:    2463892 kB
Buffers:          102596 kB
Cached:          2041736 kB
SwapCached:            0 kB
[2] 49286
VmRSS:      2024 kB
[2]+  Terminated              sleep 300
root@ubuntu-4gb-hel1-1:~/os-lab1#
```

Figure 9: free -h, /proc/meminfo, and VmRSS of a bare sleep.

Observe

(1) **Total / free RAM:** ~3.7 GiB total, ~449 MiB reported “free,” but ~2.3 GiB “avail able” (free plus reclaimable cache) – the OS uses most of the rest as disk cache (buff/cache ≈ 2.2 GiB) rather than leaving it idle.

(2) **What swap is:** disk space the OS uses as overflow for RAM, so inactive pages can be written out to free physical RAM for active use. On this VM swap is 0B – there is no overflow configured, so if RAM runs out the kernel’s OOM killer starts terminating processes instead of swapping.

(3) **VmRSS of a bare sleep:** ~2.0 MB (2024 kB). Small but expected – sleep is a tiny dynamically linked binary, and most of its footprint is the loader and shared libraries (libc, etc.) that are mapped in but shared with every other process already using them.

7 Part 4 – Devices and Storage

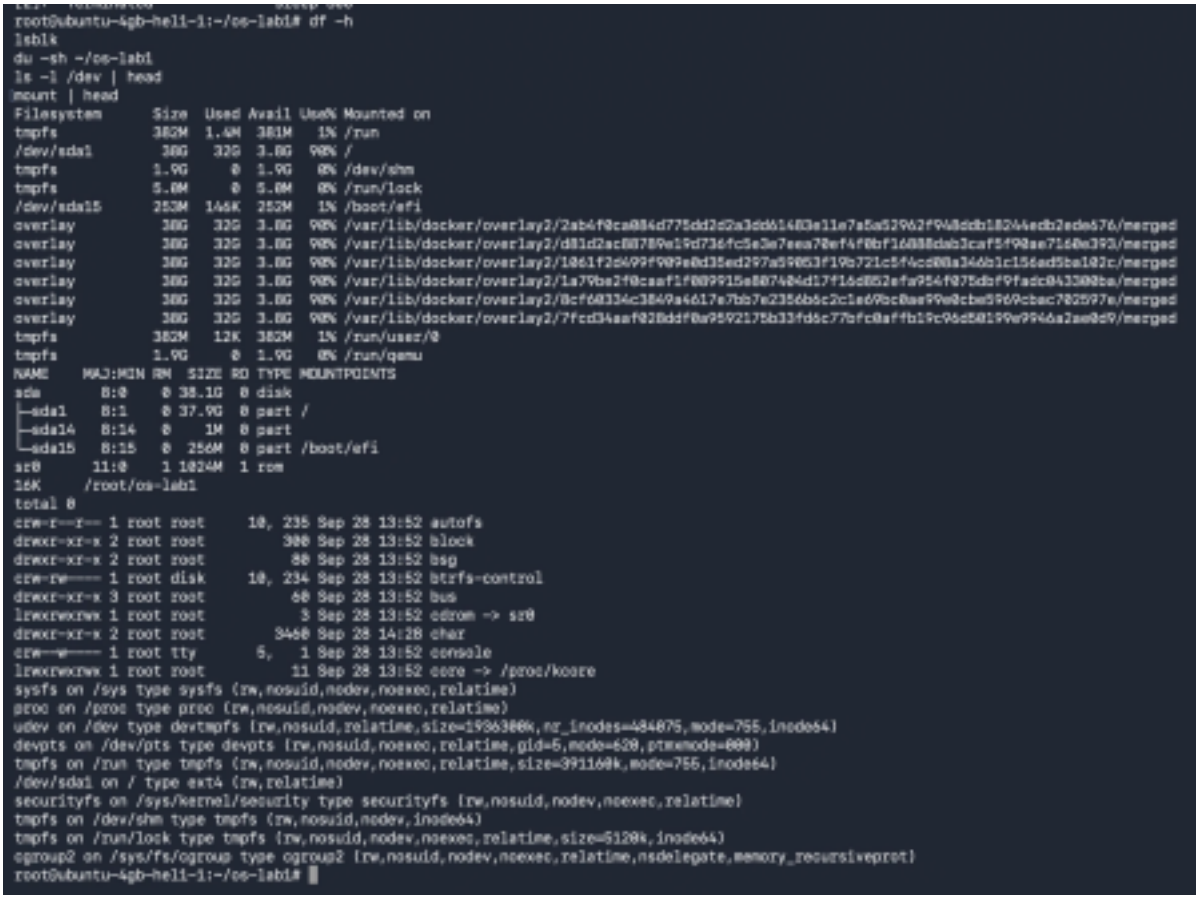


Figure 10: df -h, lsblk, du -sh, ls -l /dev, mount.

Observe

(1) **Root filesystem device:** /dev/sda1, an ext4 partition on the single 38.1 GB virtual disk sda.

(2) **One /dev entry:** /dev/sda1 – the block-device file representing the root partition of the virtual disk attached to this VM; the kernel talks to the disk hardware through this file (via the filesystem layer, not by hand).

(3) **"Everything is a file" in one sentence:** wildly different things – a physical disk partition (/dev/sda1), a live process's memory and metadata (/proc/<pid>/...), and a plain text note (note.txt) – are all accessed through the same small set of system calls (open, read, write, close), so programs need no special-case code per resource kind.

8 Bonus: PID 1

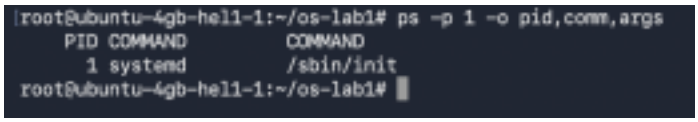


Figure 11: ps -p 1 -o pid,comm,args.

9 Conclusion and Critical Analysis

Summary of Work:

- Navigated the filesystem tree and manipulated files and permissions with ls, cp, mv, rm, and chmod.
- Listed, backgrounded, inspected, and terminated processes with ps, jobs, kill, and /proc.
- Measured total and per-process memory usage with free -h and /proc/meminfo/VmRSS. •

Inspected mounted filesystems, block devices, and device files under /dev. **Critical**

Analysis:

- **Shell semantics matter:** two real mistakes made during this lab (mkdir && cd silently no-op-ing after a failure, and typing a documentation placeholder <PID> literally) were more instructive than the commands that worked on the first try – they show that the shell executes exactly what is typed, not what is intended.
- **“Available” vs. “free” memory:** a beginner reading only the “free” column of free -h would wrongly conclude the VM is almost out of RAM, when in fact most of the missing memory is reclaimable disk cache.
- **Process-state snapshots are timing-sensitive:** the exact State: value read from /proc/<pid>/status depends on the precise instant the snapshot is taken relative to scheduling, a reminder that /proc exposes a live, constantly-changing kernel data struc ture rather than a static log.

Overall, this laboratory work demonstrates, from the outside, how a real operating system manages the four classic resources – files, processes, memory, and devices – and how a small set of Unix commands is enough to observe all four directly.